

Purpose

Measure PIC16F883 instruction timing, use instruction cycle calculations to create precise software delays, extend those delays with nested loops, & then refactor the working delay into reusable subroutines.

Standards & references

- RCET 3375 Lab standard
- RCET PIC-AS style guide
- PIC16F883 Data sheet
- PIC micro Mid-Range MCU Family Reference Manual
- DLG7137 datasheet

Equipment & materials

- MPLab X IDE & pic-as toolchain
- PICKit programmer/debugger
- PIC16F883 circuit from lab 2
- 4MHz crystal oscillator circuit
- DLG7137 display
- Oscilloscope
- Frequency counter
- Logic analyzer Optional
- DMM
- Lab Book

Part 1 - Measure Instruction TimingGoal

use the PortB counter program from Lab 2 to measure the execution time of the PIC16F883 instructions.

use the RBO output waveform to:

- Measure pulse width (PW), pulse space (PS), period, & frequency
- relate the measured waveform to the instructions executing in the program loop
- determine the actual instruction cycle time, T_{cy}
- Calculate the oscillator frequency
- Verify the effect of adding a NOP instruction

The purpose of this Lab is to connect assembly code to real time.

Before Lab

Start with the working PortB counter program from Lab 2.

Review the instructions executed repeatedly in the main loop

For each instruction in the repeating path:

1. Identify how many instruction cycles it requires
 2. Determine how many instruction cycles occur between changes of RBO
- The PIC instruction clock is related to the oscillator by:

$$T_{osc} = 1/F_{osc}$$

$$T_{cy} = 4 \cdot T_{osc}$$

$$F_{osc} = 4/T_{cy}$$

ART
9/8/26

Experiment 3 Delays

The circuit (Figure 1 on pg 145) uses a nominal 4MHz crystal. Calculate the expected instruction cycle time before making measurements.

- Determine the expected number of instruction cycles for
- RBO PW & PS
 - One complete period

Record your predictions

Using the exact same code from Lab 2 pt2 the increment for RBO should take 3 instruction cycles

Table
1-1

TABLE 15.2: PIC16F882/883/884/886/887 INSTRUCTION SET

Mnemonic, Operands	Description	Cycles	14-Bit Opcode				Status Affected	Notes	
			MSb		LSb				
BYTE-ORIENTED FILE REGISTER OPERATIONS									
ADDWF	f, d	Add W and f	1	00	0111	dfff	ffff	C, DC, Z	1, 2
ANDWF	f, d	AND W with f	1	00	0101	dfff	ffff	Z	1, 2
CLRF	f	Clear f	1	00	0001	1fff	ffff	Z	2
CLRW	-	Clear W	1	00	0001	0xxx	xxxx	Z	
COMF	f, d	Complement f	1	00	1001	dfff	ffff	Z	1, 2
DECf	f, d	Decrement f	1	00	0011	dfff	ffff	Z	1, 2
DECFSZ	f, d	Decrement f, Skip If 0	1(2)	00	1011	dfff	ffff		1, 2, 3
INCF	f, d	Increment f	1	00	1010	dfff	ffff	Z	1, 2
INCFSZ	f, d	Increment f, Skip If 0	1(2)	00	1111	dfff	ffff		1, 2, 3
IORWF	f, d	Inclusive OR W with f	1	00	0100	dfff	ffff	Z	1, 2
MOVF	f, d	Move f	1	00	1000	dfff	ffff	Z	1, 2
MOVWF	f	Move W to f	1	00	0000	1fff	ffff		
NOP	-	No Operation	1	00	0000	0xx0	0000		
RLF	f, d	Rotate Left f through Carry	1	00	1101	dfff	ffff	C	1, 2
RRF	f, d	Rotate Right f through Carry	1	00	1100	dfff	ffff	C	1, 2
SUBWF	f, d	Subtract W from f	1	00	0010	dfff	ffff	C, DC, Z	1, 2
SWAPF	f, d	Swap nibbles in f	1	00	1110	dfff	ffff		1, 2
XORWF	f, d	Exclusive OR W with f	1	00	0110	dfff	ffff	Z	1, 2
BIT-ORIENTED FILE REGISTER OPERATIONS									
BCF	f, b	Bit Clear f	1	01	00bb	bfff	ffff		1, 2
BSF	f, b	Bit Set f	1	01	01bb	bfff	ffff		1, 2
BTFSC	f, b	Bit Test f, Skip if Clear	1 (2)	01	10bb	bfff	ffff		3
BTFSS	f, b	Bit Test f, Skip if Set	1 (2)	01	11bb	bfff	ffff		3
LITERAL AND CONTROL OPERATIONS									
ADDLW	k	Add literal and W	1	11	111x	kkkk	kkkk	C, DC, Z	
ANDLW	k	AND literal with W	1	11	1001	kkkk	kkkk	Z	
CALL	k	Call Subroutine	2	10	0kkk	kkkk	kkkk		
CLRWDI	-	Clear Watchdog Timer	1	00	0000	0110	0100	$\overline{TO}, \overline{PD}$	
GOTO	k	Go to address	2	10	1kkk	kkkk	kkkk		
IORLW	k	Inclusive OR literal with W	1	11	1000	kkkk	kkkk	Z	
MOVLW	k	Move literal to W	1	11	00xx	kkkk	kkkk		
RETFIE	-	Return from interrupt	2	00	0000	0000	1001		
RETLW	k	Return with literal in W	2	11	01xx	kkkk	kkkk		
RETURN	-	Return from Subroutine	2	00	0000	0000	1000		
SLEEP	-	Go into Standby mode	1	00	0000	0110	0011	$\overline{TO}, \overline{PD}$	
SUBLW	k	Subtract w from literal	1	11	110x	kkkk	kkkk	C, DC, Z	
XORLW	k	Exclusive OR literal with W	1	11	1010	kkkk	kkkk	Z	

Main:

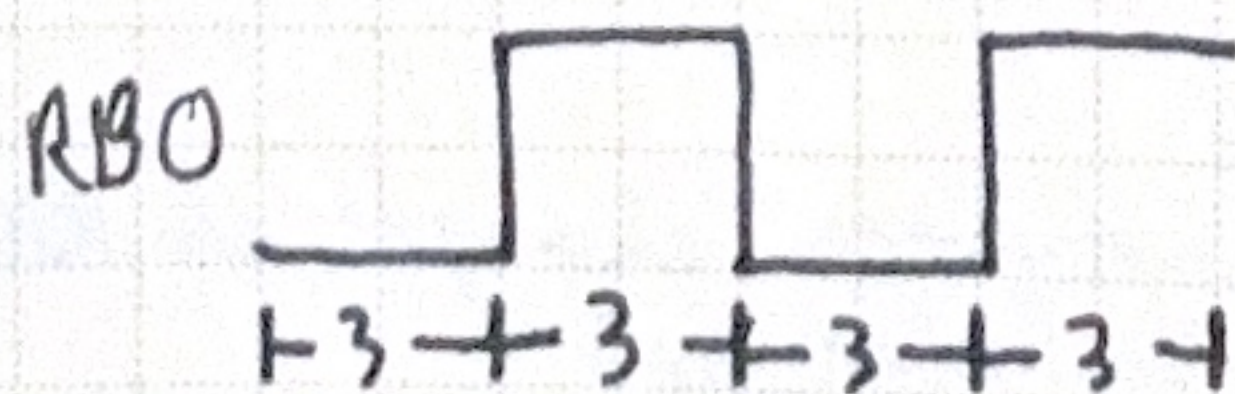
// main goes here

INCF 0x06, 1 ; Bank 0 is already active, increment portB on a loop
goto Main

End

PortB Increment Loop Code 1-1

with INCF & GOTO there are 3 instruction cycles.



$$T_{osc} = 1/4\text{MHz}$$

$$T_{cy} = 250\text{ns} \cdot 4$$

$$F_{osc} = 4/T_{cy}$$

$$T_{cy} = 1\mu\text{s per cycle}$$

6 cycles for full period

PW = 3μs	PS = 3μs	Period = 6μs
----------	----------	--------------

Predictions for RBO

ART
9/8/26

Experiment 3 Delays

ART
9/8/26

In the lab

1. Measure the Lab 2 Counter. Build & program the PIC16F883 using your working Lab 2 PortB Counter program.

Connect the oscilloscope to RBO, the Least Significant bit of the Counter output.

measure & record: PW, PS, Period, frequency

Capture an O-scope image showing enough of the waveform to verify the measurements

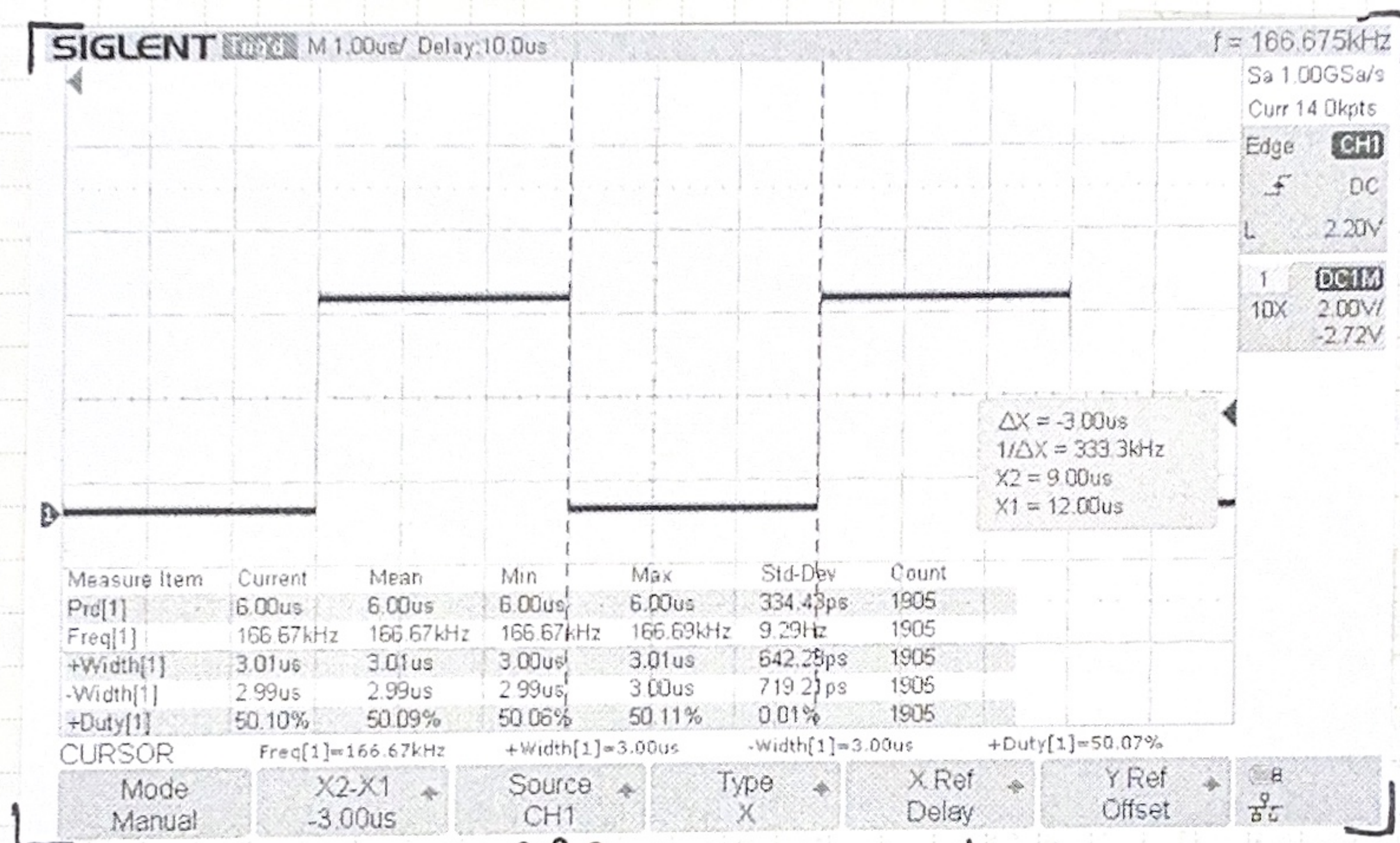


Figure 1-1 RBO Pulse Output

as PortB increments RBO should drop high & low at a rate equal to the number of instruction cycles in μs

2. Determine the instruction cycle time

$N = \text{instruction cycles}$ $T_{cy} = \frac{PW}{N} = \frac{3\mu s}{3} = 1\mu s$

3. Determine the Oscillator frequency

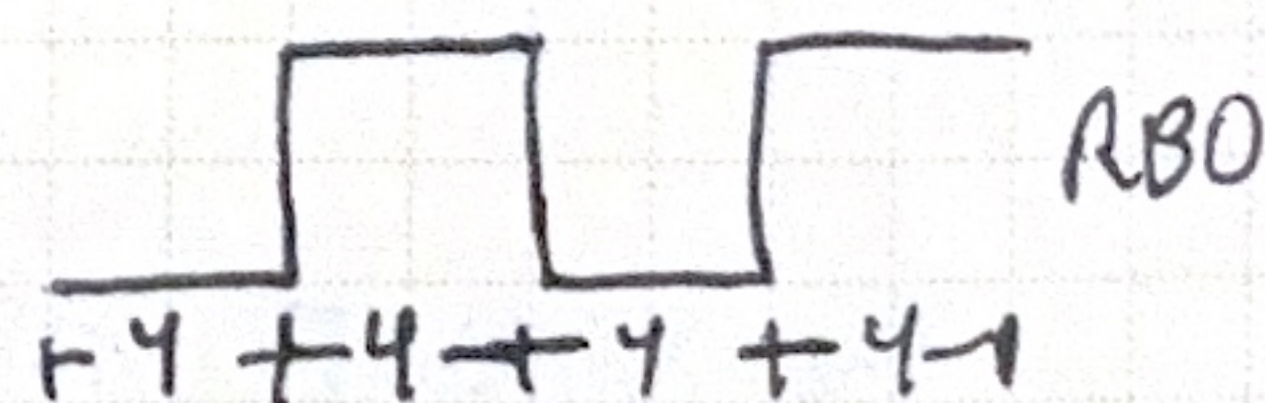
$F_{osc} = \frac{1}{T_{cy}} = \frac{1}{1\mu s} = 1\text{MHz}$

4. Add & remove NOP instruction to the repeating Loop & predict the PW, PS, Period, & frequency. Then remeasure & compare the difference

Adding an NOP adds 1 instruction cycle

$T_{osc} = 1\mu s$ $T_{cy} = 250ns \cdot 4$ $F_{osc} = 1/T_{cy}$

$T_{cy} = 1\mu s \text{ per cycle}$



8 cycles for period

$PW = 4\mu s$ $PS = 4\mu s$ $Period = 8\mu s$ $freq = 125\text{kHz}$

ART 161
9/8/26

Experiment 3 Delays

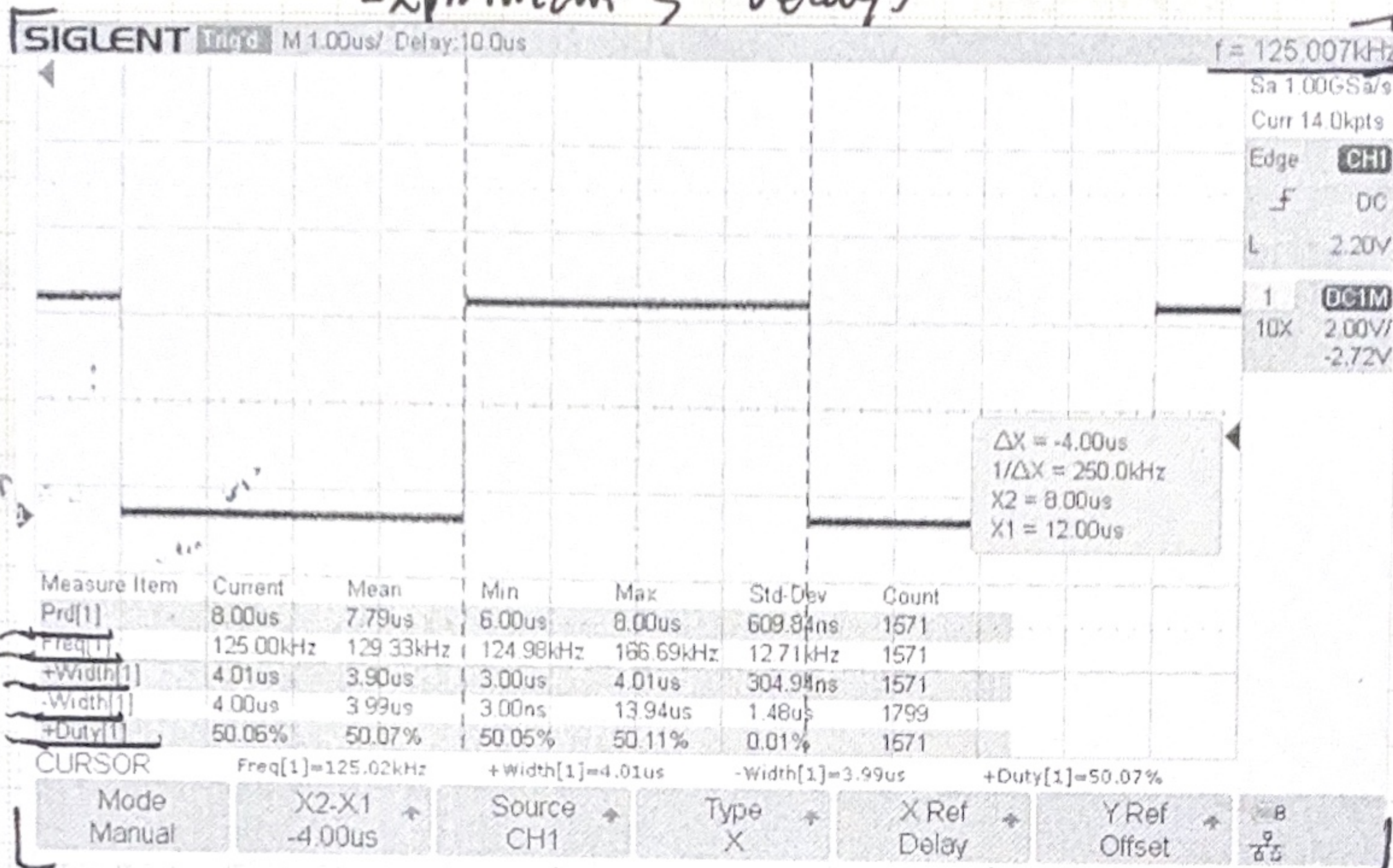


Figure 1-2 RBO Pulse output with NOP

TJR
9-9-26

*flow chart on pg 148

Part 2 - Create a Precise Short Delay

Goal

Create a new PIC16F88 project that toggles RBO using XORF, then add a calculated software delay to produce a 10kHz square wave. Use a nominal $T_{CY} = 1\mu s$

Before Lab

Create a new MPLabX project for this part. Use base code from your first project.

Begin with this repeating mainloop

Main:

```
// main goes here
MOVLW 0x01      ; 1 TCY
XORWF 0x06,1    ; 1 TCY
goto Main      ; 2 TCY
```

XOR Code 2-1

4 instruction cycles

This code should produce a 4us PW square wave @ 125kHz

PW=4us PS=4us Period=8us frequency=125kHz
Duty cycle=50%

The XORWF with one of its pins, the w pin, tied high acts as an inverter. So every loop changes RBO's state

RBO=1 $\Rightarrow$ new RBO=0
w=1 $\Rightarrow$

Experiment 3 Delays

ART
9/9/26

In the Lab

1. Verify the XORWF toggle loop
Using frequency counter the loop measures 125.0066KHz

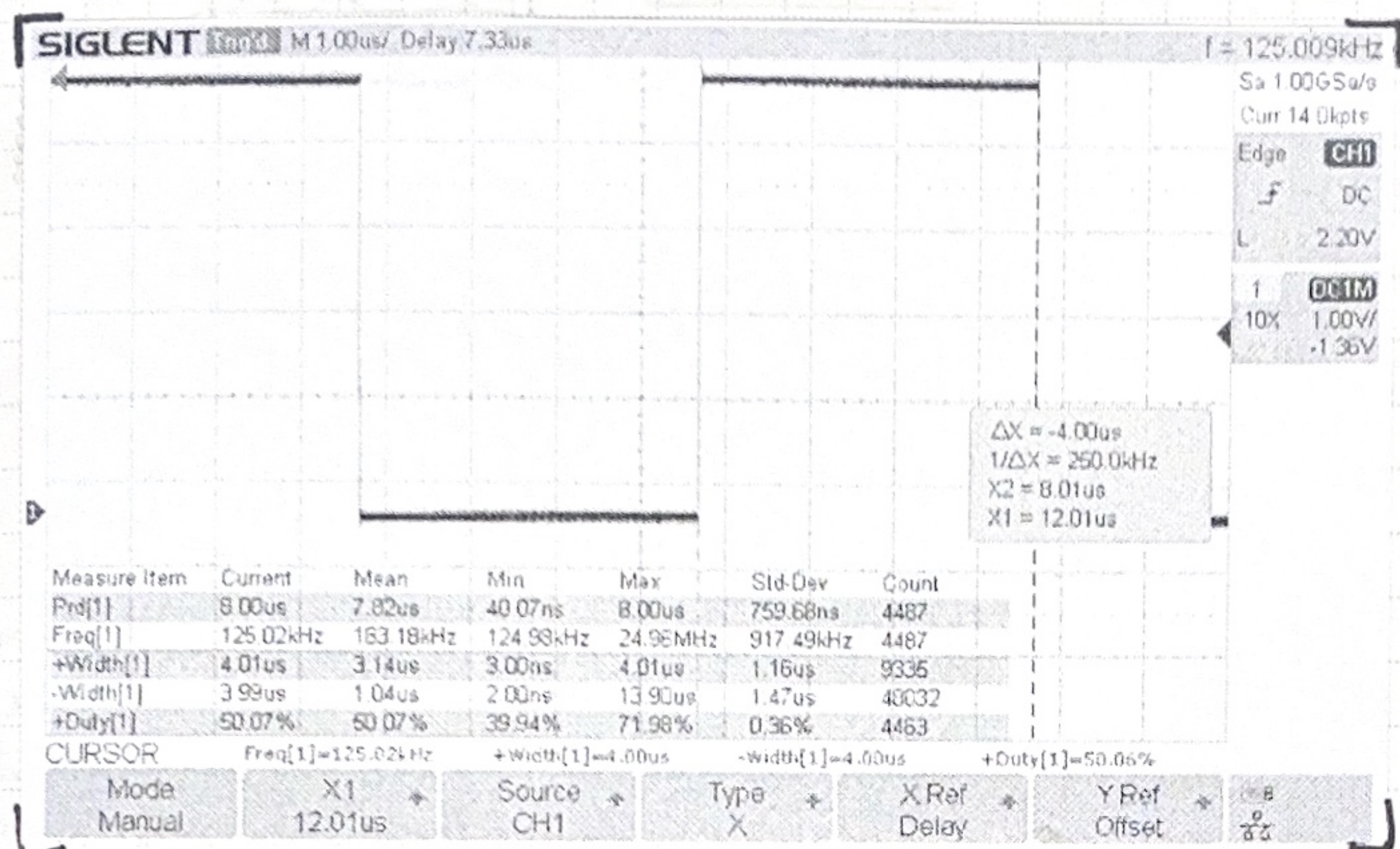


Figure 2-1

XORWF loop measurements on RBO

Counter F = 125.0066KHz | O-scope F = 125.02KHz

The counter has higher resolution & Accuracy

2. Determine the required timing for a 10kHz Pulse @ 50% Duty cycle

$$\text{Period} = 1/10k = 100\mu s \quad \text{PW} = 50\mu s \quad \text{PS} = 50\mu s$$

Design a delay loop with a total main loop $T_{cy} = 50\mu s$

3. Create a software delay loop

count EQU 0x20 ; use data memory address 0x20 as an 8-bit loop counter

Main:

MOVLW 0x01 ; 1 TCY code not part of delay
XORWF 0x06,1 ; 1 TCY code not part of delay

MOVLW 0x09 ; 1 TCY delay overhead $\leftarrow N=9$ in this case
MOVWF count ; 1 TCY delay overhead

Delay:

DECFSZ count,1 ; 1(2) 1 TCY if not zero, 2 TCY if zero
goto Delay ; 2 TCY

goto Main ; 2 TCY code not part of delay

End

Delay loop code 2-2

} Main loop XORWF + 2TCY

} Delay overhead + 2TCY

} Delay loop $(N(1+2)-1)+2$

goto main

$$T_{delay} = 3N + 2 - 1 = 3N + 1 \leftarrow \text{does not include main code}$$

$$T_{delay} + \text{Goto main} + \text{XORWF Code} = \boxed{PW = 3N + 5}$$

$$\text{we want } PW = 50\mu s = 3N + 5 \rightarrow 45 = 3N \quad \boxed{N=15}$$

N = number of loops through delay

ART
9/9/26

Experiment 3 Delays

4. Determine Counter value

N Counter = 15

$$PW = 3N + 5 = 3(15) + 5$$

$$freq = 10K$$

$$PW = 50\mu s$$

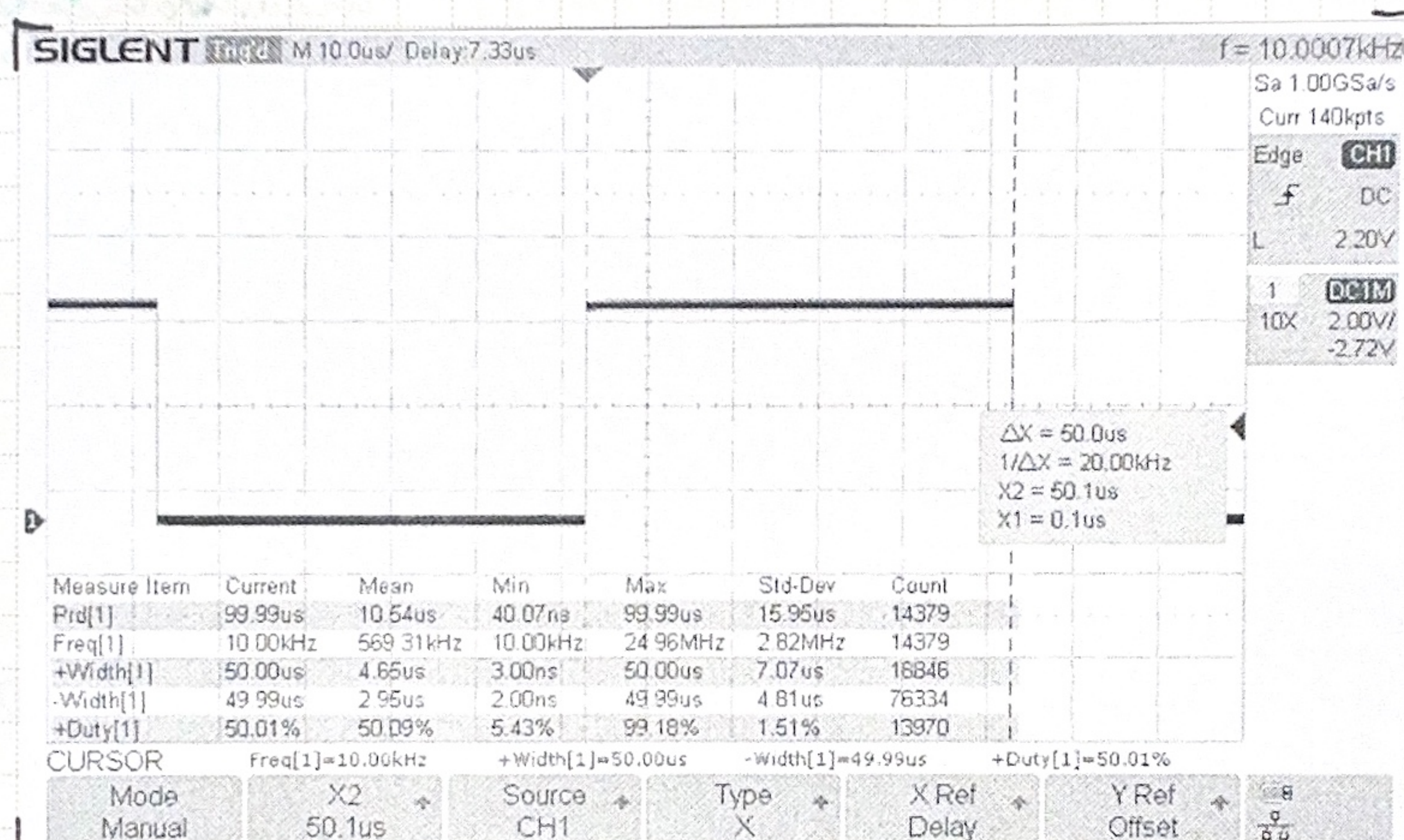
$$PS = 50\mu s$$

$$Period = 100\mu s$$

Duty cycle = 50%

5. Build & measure the delay

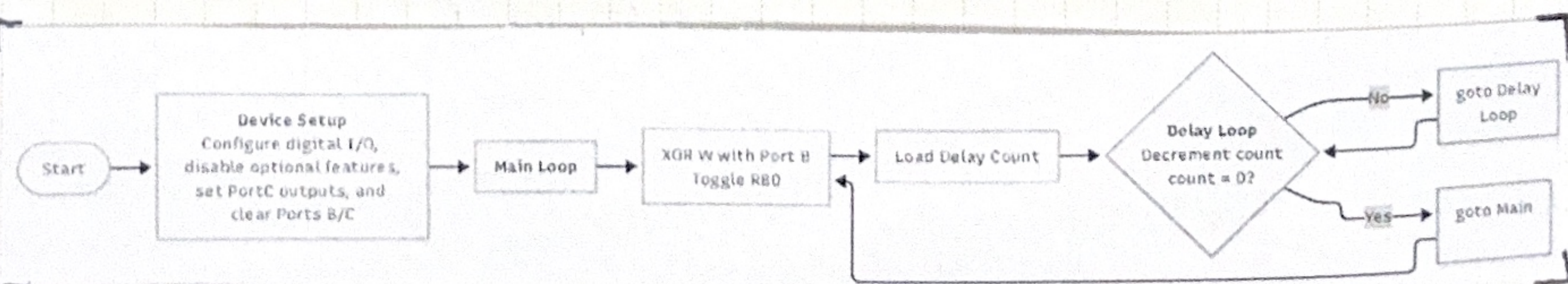
freq counter = 10.00052 KHz O-scope = 10.0007 KHz



Quantity	Predicted at 1 us/TCY	Expected Using Measured TCY	Oscilloscope	Frequency Counter
PW	50us	50.125us	50us	50.008us
PS	50us	50.125us	49.999us	49.986us
Period	100us	100.250us	99.99us	99.995us
Frequency	10kHz	9.975 kHz	10.0007kHz	10.00052 kHz
Duty cycle	50%	50%	49.99%	50.012%

SFRs in Setup just configure port B like Lab2 pt2

We also used a general register 0x20 as a Count loader for the Delay using the EQU function



FC1 - 50us Delay Loop flowchart

Part 3 - Create & Verify a Long DelayGoal

Use Nested Software delay Loops to create a much longer time interval than can be produced conveniently with a single 8-bit loop to Display an alternating 5 & 0 with a PW of .5 Seconds.
Must be accurate within one $T_{CY} = 1\mu s$ explain if this can be measured with current lab equipment

Before Lab

With $T_{CY} = 1\mu s$ determine the maximum delay the single loop can produce @ 8-bits from part 2.
Design a nested loop capable of a .5 Sec delay.

Main:

```
MOVLW 0x01      ;1 TCY Set the bit to be toggled by XOR
XORWF 0x06,1    ;1 TCY Toggle RBO

MOVLW 0x05      ;1 TCY set the lower bits to 5. Upper bits unaffected by XOR
XORWF 0x07,1    ;1 TCY toggle between 5 & 0

MOVLW 0x0F      ;1 TCY delay overhead
MOVWF mainCount ;1 TCY delay overhead
```

MainDelay:

```
MOVLW 0x0F      ;1 TCY delay overhead
MOVWF outCount  ;1 TCY delay overhead
```

OutDelay:

```
MOVLW 0x0F      ;1 TCY delay overhead
MOVWF inCount   ;1 TCY delay overhead
```

InDelay:

```
DECFSZ inCount,1 ;1(2) 1 TCY if not zero, 2 TCY if zero
goto InDelay     ;2 TCY

DECFSZ outCount,1 ;1(2) 1TCY if not zero, 2 TCY if zero
goto OutDelay    ;2 TCY

DECFSZ mainCount,1 ;1(2) 1TCY if not zero, 2 TCY if zero
goto MainDelay   ;2 TCY

goto Main        ;2 TCY code not part of delay
```

} toggle RBO + 2TCY

} toggle 5 & 0 + 2TCY

} Main overhead + 2TCY

} outer overhead + 2TCY

} inner overhead + 2TCY

} $I(2+1)-1$ } $U(3I+3)$ } $M(U(3I+3)+4)+8$

Goto RTCP

Nested delay Loop code 3-1

Equation Setup like this:

$$M \left(U \left(I(2+1)-1 + 2 + (2+1)-1 + 2 + (2+1)-1 + 2 + 2 + 2 + 2 \right) + 4 \right) + 8$$

outer + overhead
inner + overhead
math + overhead

$$M(U(3I+3)+4)+8 = 500ms \text{ or } 500,000\mu s$$

$$24(53(3(130)+3)+4)+8 = 500,000$$

There is any number of combinations of M, U, & I
That could get close to 500,000 with very few nops.
These are the numbers I chose to use

$$M=24 \quad U=53 \quad I=130$$

* This was also given to AI which gave exact numbers for the variables to land exactly on 500,000 $M=24 \quad U=53 \quad I=130$

Toggle RBO
Toggle 5 & 0
Goto main

Experiment 3 Delays

ART
9/16/26

In the Lab

1. Build the nested delay
2. Measure with the O-Scope

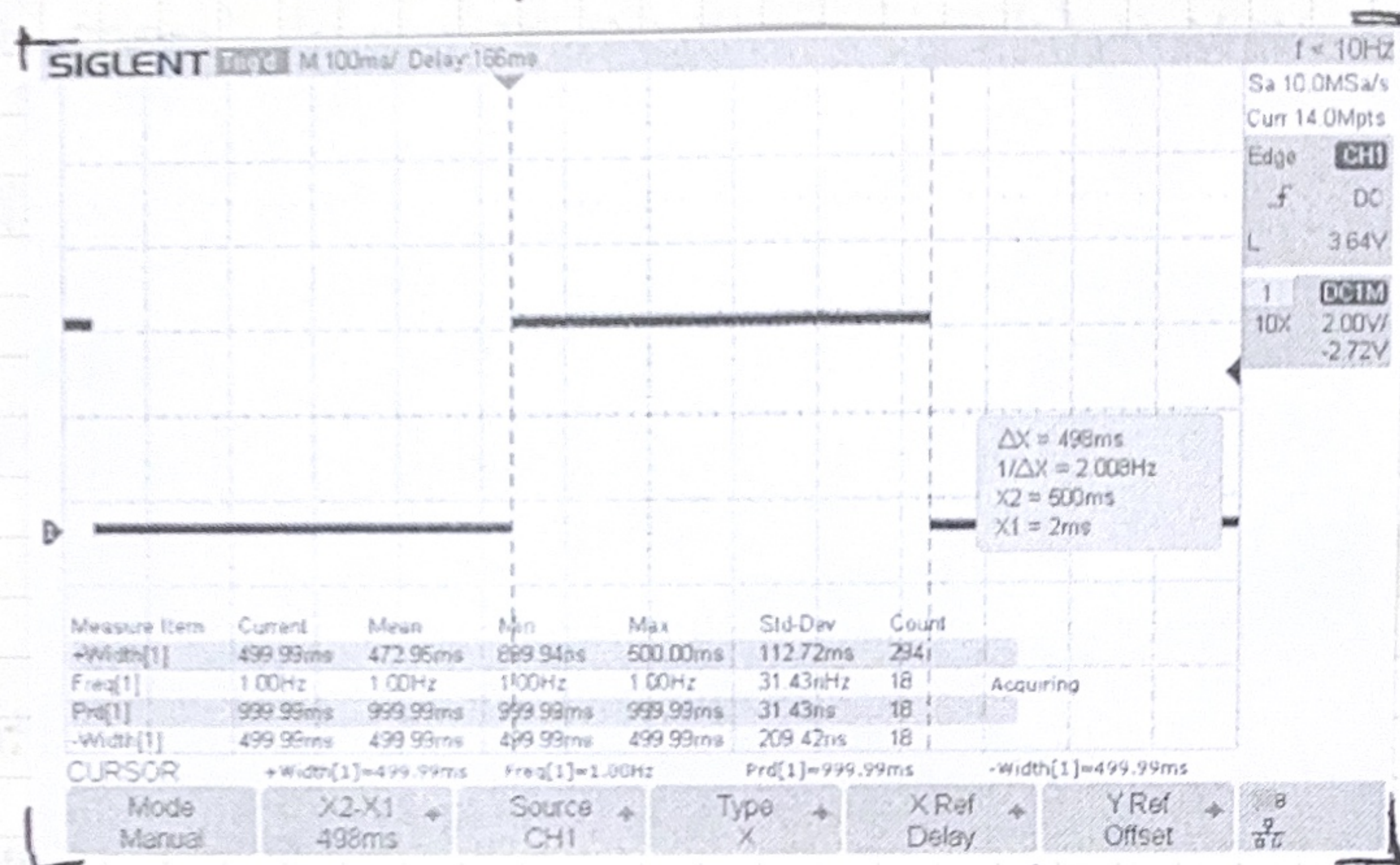


Figure 3-1 O-Scope Capture of Nested Delay

* The resolution of this capture isn't nearly enough to tell if we are within 1μs of 500,000μs

3. Measure with the Frequency Counter

frequency Counter = 501,241μs = PW

The counter, with some sensitivity adjustments, has the resolution to measure down to 1 Tcy.

Obviously the counter is showing the wrong/unwanted number. After swapping the counter to see if it was the problem & seeing no change I came to the conclusion it was the numbers I chose for the variables & therefore my equation is wrong.

Resolve equation: I = innerLoop Count U = outerLoop Count M = Mainloop Count

$$\underbrace{3(I-1)+2}_{\text{non Loop Loop}} \rightarrow 3I-1 \rightarrow \underbrace{3I-1+2}_{\text{Loop overhead}}$$

$$\underbrace{(U-1)(3+(3I+1))}_{\text{non Loop Loop}} + \underbrace{(2+(3I+1))}_{\text{Loop}} \rightarrow U(3I+4) - (3I+4) + (3I+3) \rightarrow \underbrace{U(3I+4)-1+2}_{\text{Loop overhead}}$$

$$\underbrace{(M-1)(3+(U(3I+4)+1))}_{\text{non Loop Loop}} + \underbrace{(2+U(3I+4)+1)}_{\text{Loop}} \rightarrow M(U(3I+4)+4) - (U(3I+4)+4) + (U(3I+4)+3)$$

$$\boxed{M(U(3I+4)+4)+7}$$

$$\underbrace{M(U(3I+4)+4)-1+8}_{\text{Loop overhead + Main code}}$$

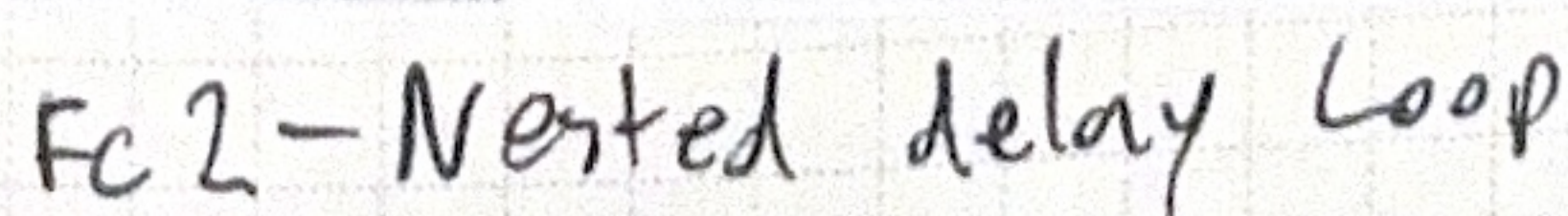
NOP $M=6$ $U=168$ $I=164$

$$(24)(153)(3(136)+4)+4)+7 = 501,271 \mu s \leftarrow \text{Exactly the same distance from the measured distance with new numbers}$$

New numbers need no NOP $M=6$ $U=129$ $I=214$

4. Change the program by one Tcy

TSR
9-11-26



Part 4 - Delay Subroutines & the Hardware Stack

ART
9/11/26Goal

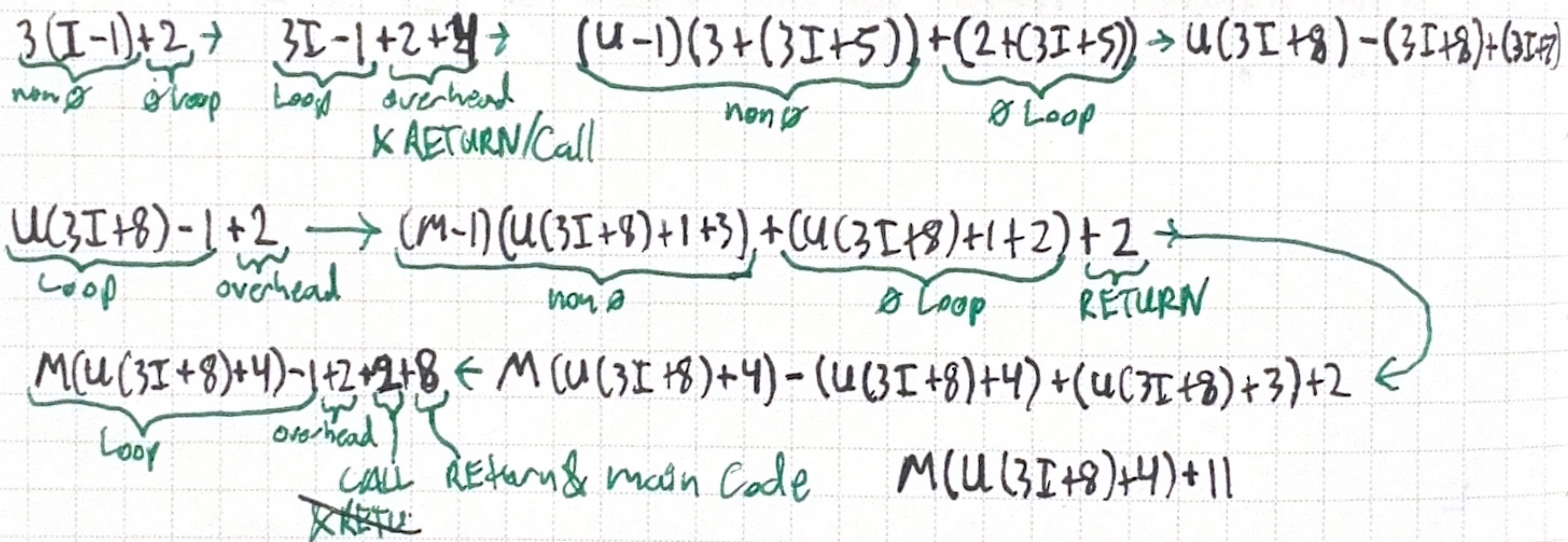
Refactor the working Nested delay from Part 3 into reusable Subroutine using CALL & RETURN

Before Lab

Review the subroutine, Program Counter, & hardware stack Concepts

modify the part 3 code so it rotates 5 & 8 on Display & has a call for a large delay & inside another call for a smaller Delay.

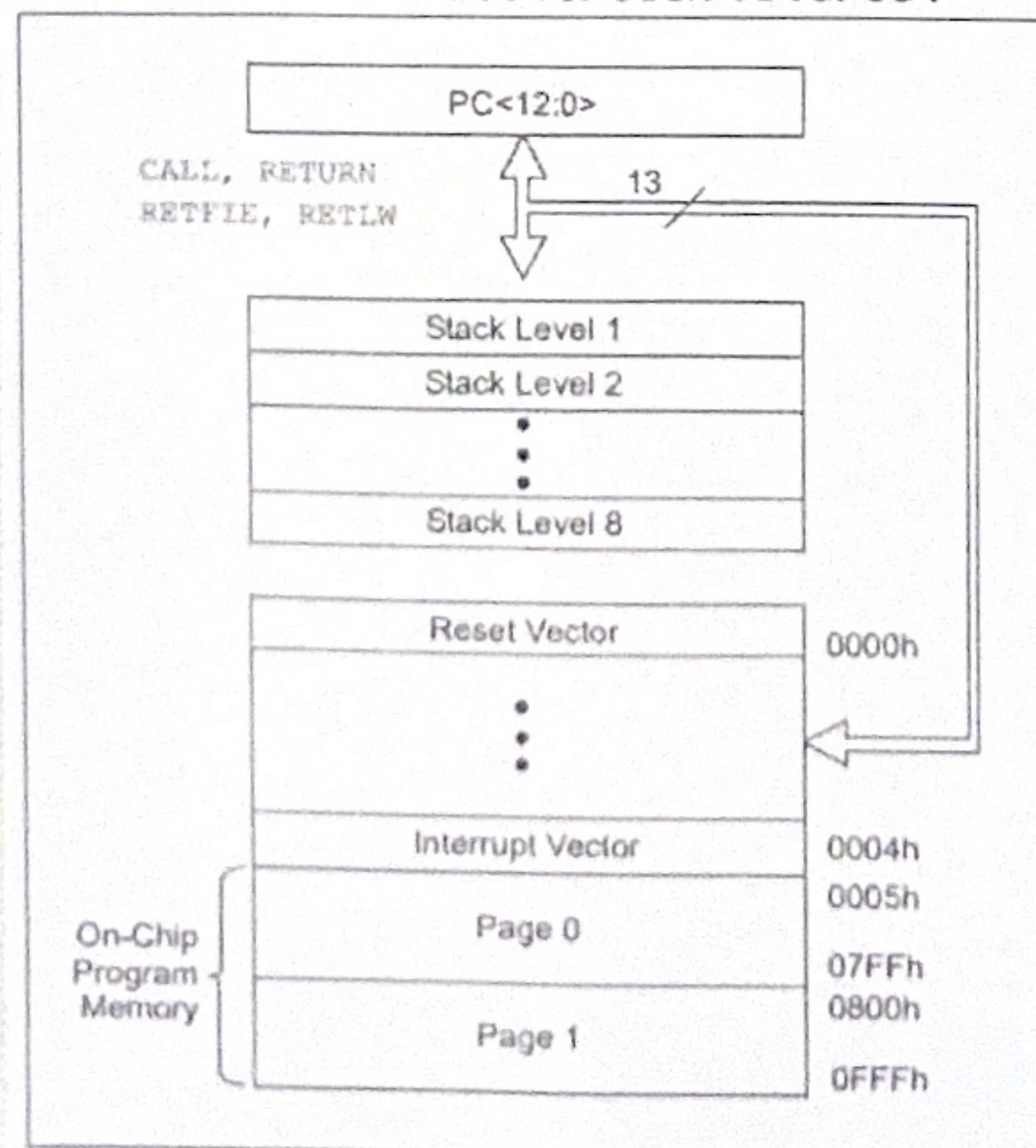
Create an updated flow chart & recalculate the timing to have the PW = .5 Seconds



* we still must account for the 35ns crystal imperfection

$$M=8 \quad U=249 \quad I=81$$

FIGURE 2-2:
4-1 PROGRAM MEMORY MAP
AND STACK FOR THE
PIC16F883/PIC16F884



The program memory stack is 8 levels deep. When a call instruction is used the PC+1 is pushed to the stack. If another call is used before a Return, like in my code, that PC+1 is pushed on top of the last. As Returns are used the PC+1 Locations are popped out in a First-in-Last-out pattern.

my code does 2 Levels deep at most

ART
9/11/26

Experiment 3 Delays

ART
9/11/26

Main:

```
MOVLW 0x01      ;1 TCY Set the bit to be toggled by XOR
XORWF 0x06,1    ;1 TCY Toggle RB0
```

NOP

```
MOVLW 0x05      ;1 TCY set the lower bits to 5. Upper bits unaffected by XOR
XORWF 0x07,1    ;1 TCY toggle between 5 & 0
```

```
CALL LargeDelayLoop ;2 TCY
```

```
goto Main      ;2 TCY
```

LargeDelayLoop:

```
MOVLW 0x08      ;1 TCY delay overhead
MOVWF mainCount ;1 TCY delay overhead
```

MainDelay:

```
MOVLW 0xF9      ;1 TCY delay overhead
MOVWF outCount  ;1 TCY delay overhead
```

OutDelay:

```
CALL SingleDelayLoop;2 TCY
```

```
DECFSZ outCount,1 ;1(2) 1TCY if not zero, 2 TCY if zero
goto OutDelay     ;2 TCY
```

```
DECFSZ mainCount,1 ;1(2) 1TCY if not zero, 2 TCY if zero
goto MainDelay    ;2 TCY
```

```
RETURN           ;2 TCY code not part of delay
```

SingleDelayLoop:

```
MOVLW 0x51      ;1 TCY delay overhead
MOVWF inCount   ;1 TCY delay overhead
```

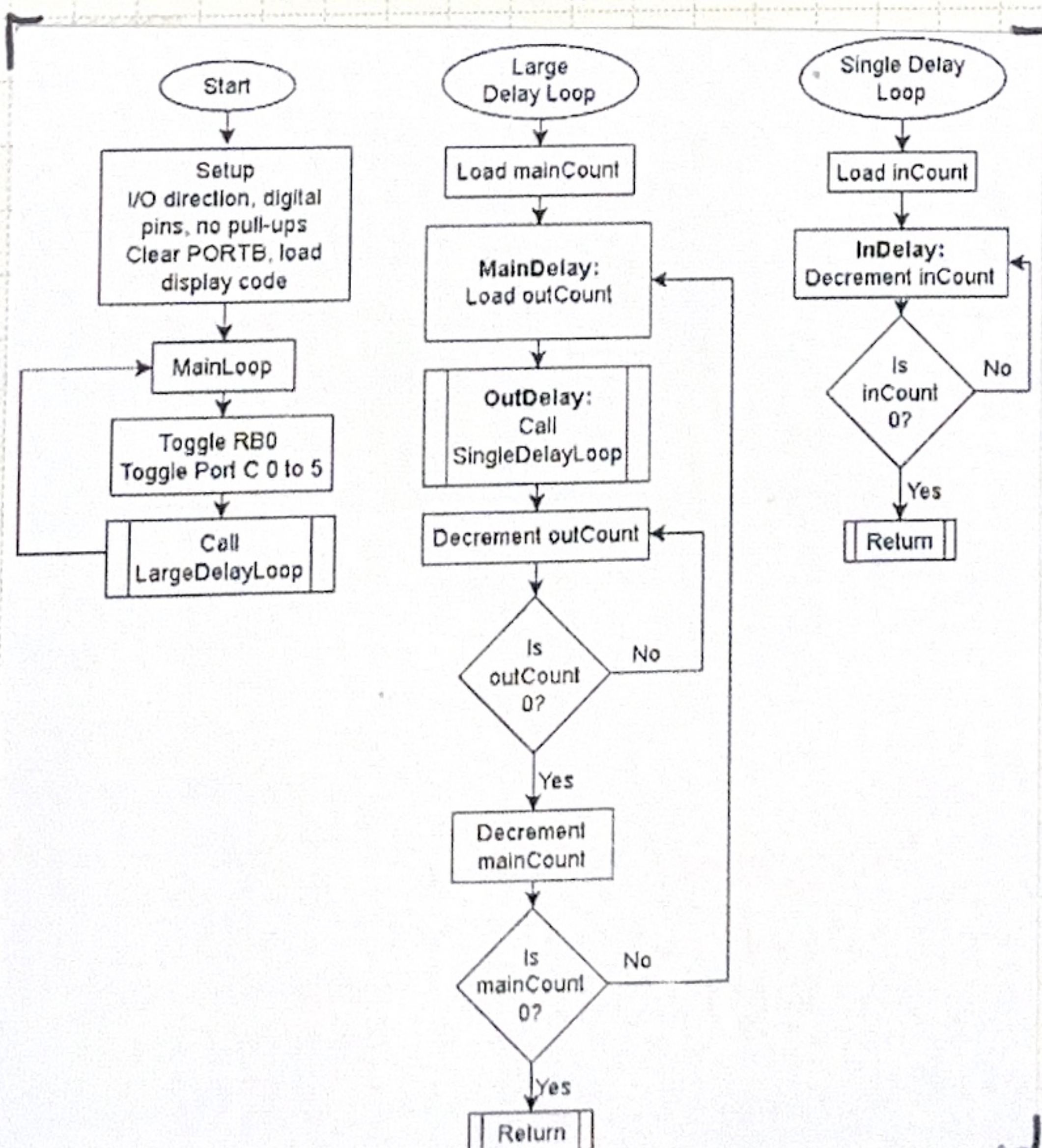
InDelay:

```
DECFSZ inCount,1 ;1(2) 1 TCY if not zero, 2 TCY if zero
goto InDelay     ;2 TCY
```

```
RETURN          ;2 TCY
```

Call Address +1
is stored on stack
Return will
use the incremented
Address

Nested Delay Loop Subroutine FC4-1 Code 4-1



Nested Delay Loop Subroutine FC4-1

ART
9/11/26
169

Experiment 3 Delays

In the Lab

1. Implement the sub routines
2. Verify the timing
3. Trace the nested calls

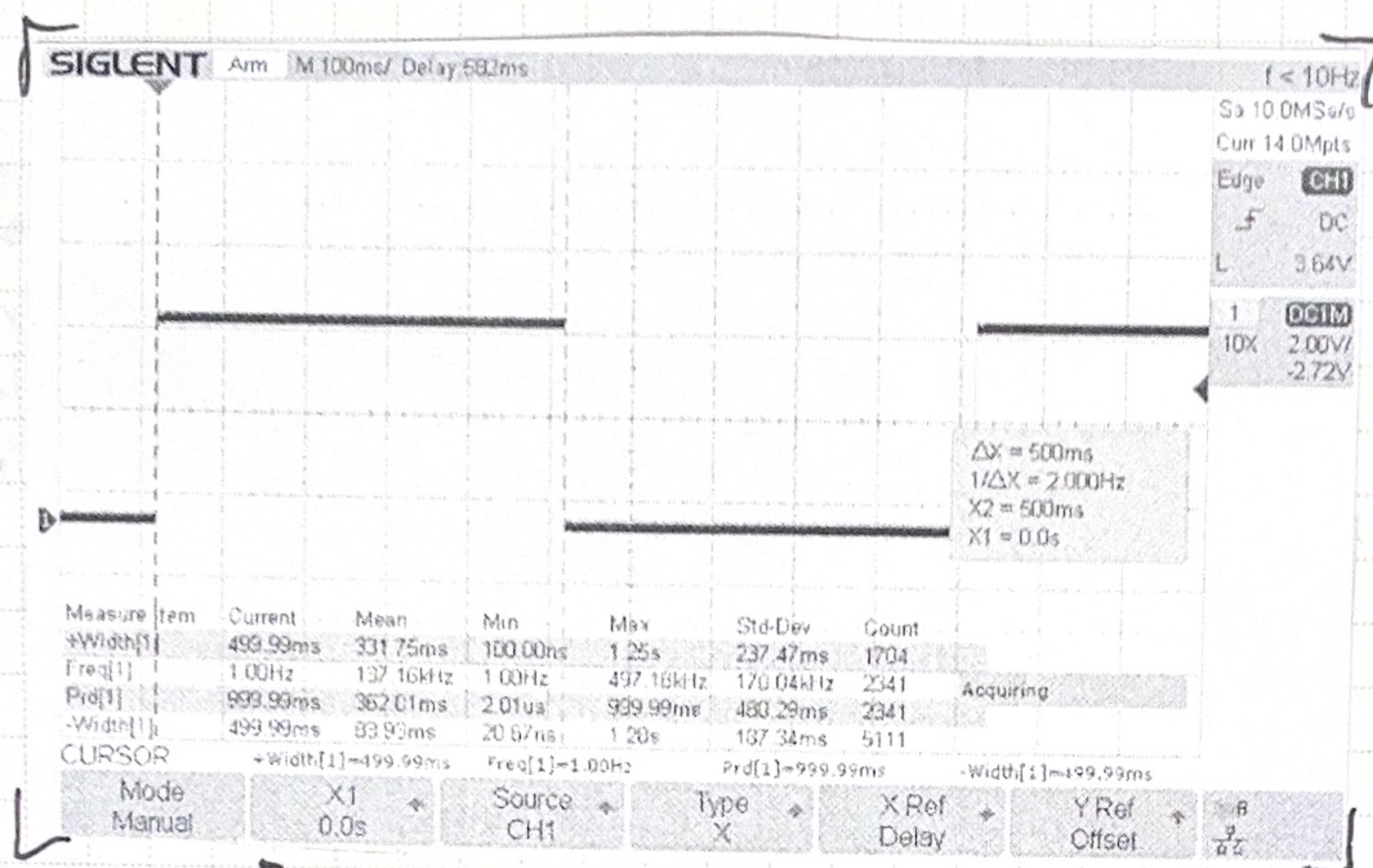


Figure 4-2 O-Scope Capture of .5sec Subroutine pulse

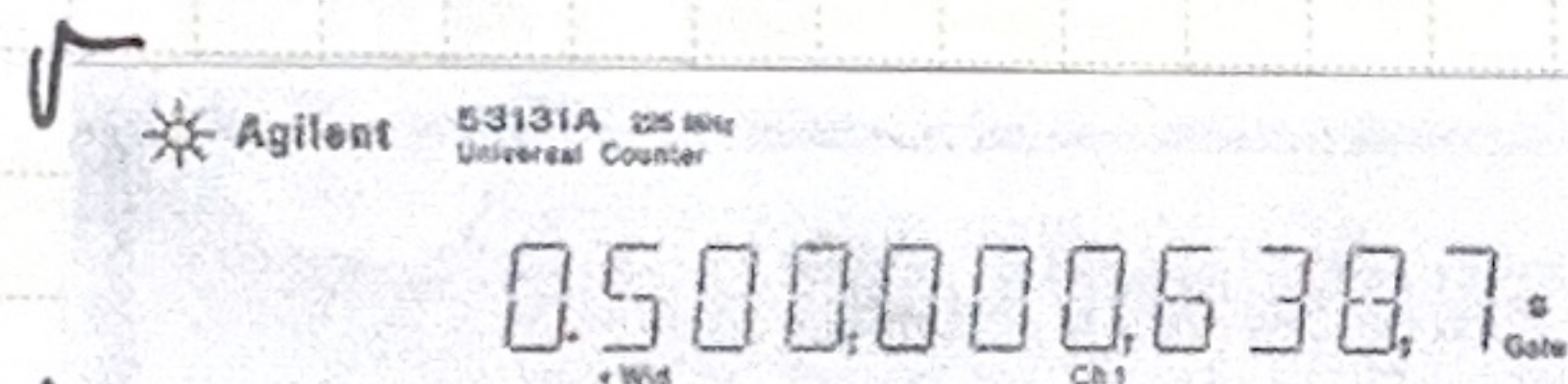


Figure 4-3 freq counter of .5sec Delay Pulse width

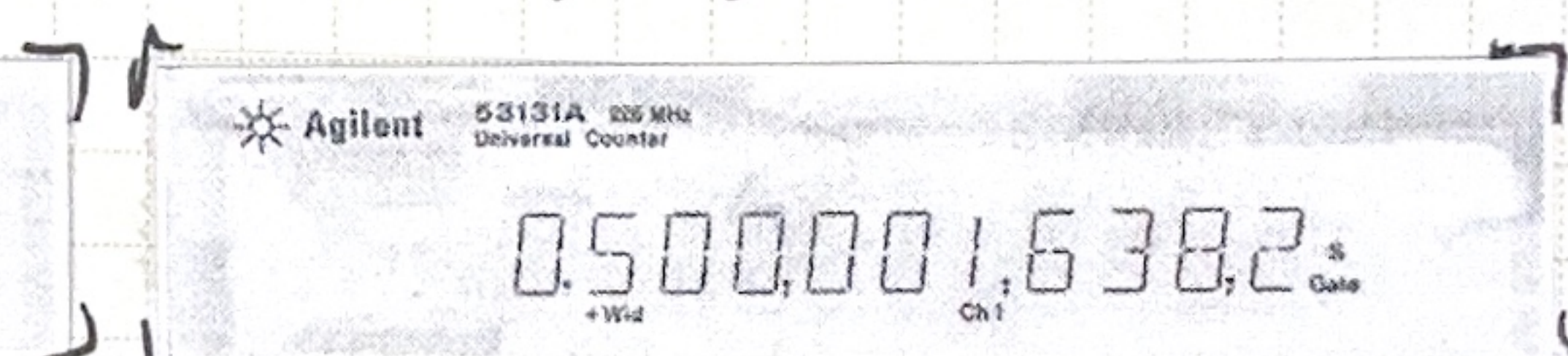


Figure 4-4 freq counter of .5sec Delay with an added NOP

TJR
9-11-26

Conclusion

This Lab Explored how to Measure, Code, Calculate, & Define Different Delay structures. The Assembly code instructions each have a time to execute associated with them. Using looping functions & relatively simple algebra, we can exactly predict how long these times take & therefore choose a preferred time. This can also be sectioned into a subroutine using the program counter stack.

AAT
9/11/26